

INSTRUCTIONS TO THE CANDIDATES

- ❖ Do not use sketch pens/ high –lighter pens/ color pencils etc., in underlining / highlighting certain points in the answer or while drawing sketches etc.,
- ❖ Check the Answer booklet thoroughly. The defective answer booklet may be returned to the invigilator before starting the examination.
- ❖ Ensure that the OMR Barcode sheet is properly secured to the answer booklet.
- ❖ Do not write on or tamper the Barcode. If you do so, your booklet will not be evaluated.
- ❖ The candidate is prohibited from writing his/ her NAME / ADDRESS/ PHONE NUMBER / EMAIL ID / Religion or any other matter on any part of the answer booklet / drawing sheet/ graph sheet/ addressing the examiner in any manner whatsoever in the answer booklet. The candidate is not supposed to tear a page in full or part of the booklet.

Non-adherence with this instruction shall result in candidate being booked under malpractice.

- ❖ Answer should be written on both sides of the paper. Candidate should write in all 25 lines in each page. It is not necessary to begin each answer in a fresh page.
- ❖ Candidate should write only question number in the margin.
- ❖ Candidate should return the answer booklet to the invigilator before leaving the examination hall.

ADDITIONAL SUPPLEMENT OF ANSWER SHEETS WILL NOT BE ISSUED.

INSTRUCTIONS TO THE EXAMINERS

- ❖ Invigilators should ensure that the above instructions are strictly followed by the candidates.
- ❖ After verifying the particulars on the OMR sheet, invigilator should sign in the box specified.
- ❖ Do not crumple or cut the sides of the sheets.
- ❖ Do not write, mark or damage the timing tracks or Barcode.

1

1.A Classifications of Design Pattern

* Structural.

* Behavioural.

*

1.C Generics.

```
Public class GenClass < DT > {
```

```
    DT Val;
```

```
    Public void setVal < DT k > {
```

```
        This.Val = k;
```

```
    }
```

```
    Public DT getVal () {
```

```
        return this.Val;
```

```
    }
```



```
Public static void main (String[] args) {
```

```
GenClass<Integer> GC = new GenClass<Integer>();
```

```
GC.setVal(10);
```

```
System.out.println(GC.getVal());
```

```
}
```

```
}
```

1.D Set & Map

Set

Map

* Hash set

* Hash map

* Linked Hash set

* Linked Hash map

* tree set

* tree map

set

```
HashSet HS = new HashSet<16, 0.75>((Integer));
```

```
HS.add(1);
```

```
HS.addAll(C);
```

```
HS.clear();
```

```
HashMap HM = new HashMap<16, 0.75>((Integer));
```

```
HM.add(1);
```

```
HM.addAll(C);
```

```
HM.addClear();
```

2

2.A Clean Coding Techniques

* Structure

* File naming

* Source File structure

* White Spaces and Indentation

* Method parameters.

* Avoid Magic coding

* Comments

* Logging

* SOLID

* DRY & KISS

2.B // Password.java

```

import java.util.Scanner;

public class Password {

    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        String Pw = Password;
        try {
            Password = sc.nextLine();
            int n = Pw.length;
            if (n > 5 & & ≤ 10)
                System.out.println ("Accepted");
            else
                System.out.println ("Not Accepted");
        } catch (err) {
            System.out.println (err);
        }
        sc.close();
    }
}

```

2.C Generic Swapping.

```

class Swapping <DT> {

```

```

    public void swap (DT[] arr, int x, int y) {
        DT A = arr[x];
        arr[x] = arr[y];
        arr[y] = A;
    }
}

```

```

public class Question {

```

```

    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int[] A = new int [5];
        String[] B = new int [10];
        for (int i = 0; i < 5; i++)
            A[i] = sc.nextInt();
        for (int i = 0; i < 10; i++)
            B[i] = sc.nextLine();
    }
}

```

```

Swapping<Integer> SI = new Swapping<Integer>();
Swapping<String> SS = new Swapping<String>();

```



```

SI. Swap (A, 2, 3);
SS. Swap (B, 1, 2);

```

```

}

```

```

}

```

2.D Comparator:

```

import java.util. ArrayList

```

```

class student {

```

```

    String Name;

```

```

    int Age;

```

```

    float GPA;

```

```

    public student (String n, int a, float g) {

```

```

        this.Name = n;

```

```

        this.Age = a;

```

```

        this.GPA = g;

```

```

    }

```

```

}

```

```

class compare GPA implements Comparator <Student> {

```

```

    public int compare (Student s1, Student s2) {

```

```

        return -(s1.GPA - s2.GPA);

```

```

    }

```

```

}

```

```

public class client {

```

```

    public static void main (String[] args) {

```

```

        ArrayList <Student> ALS = new ArrayList <Student> ();

```

```

        ALS.add (new student ("A", 19, 8));

```

```

        ALS.add (new student ("B", 20, 9.8));

```

```

        ALS.add (new student ("C", 18, 7.9));

```

```

        Collections.sort (ALS, new compare GPA);

```

```

        System.out.println (ALS);

```

```

    }

```

```

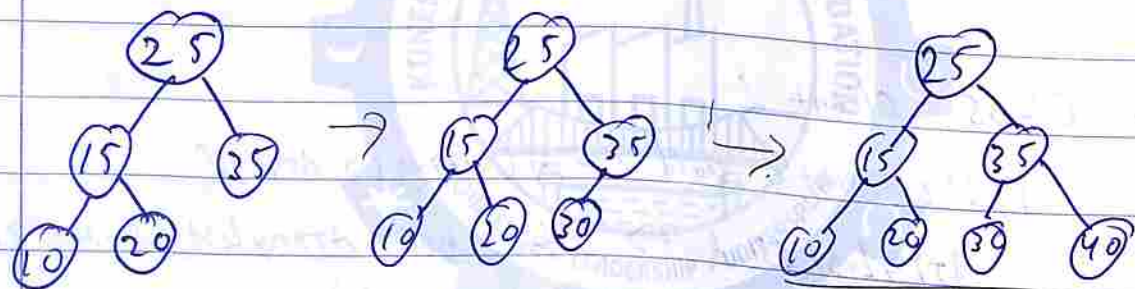
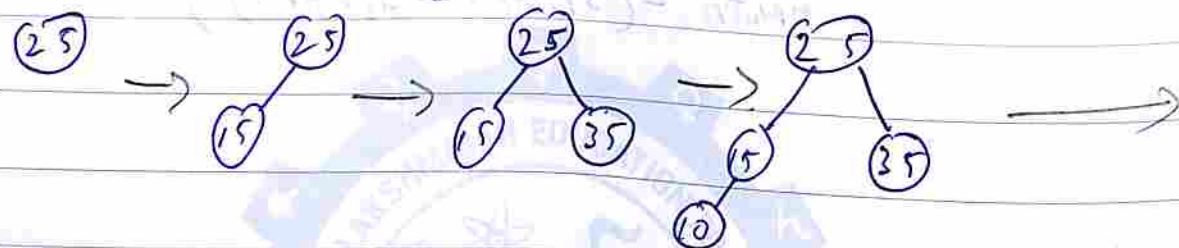
}

```


4

U.A. BST.

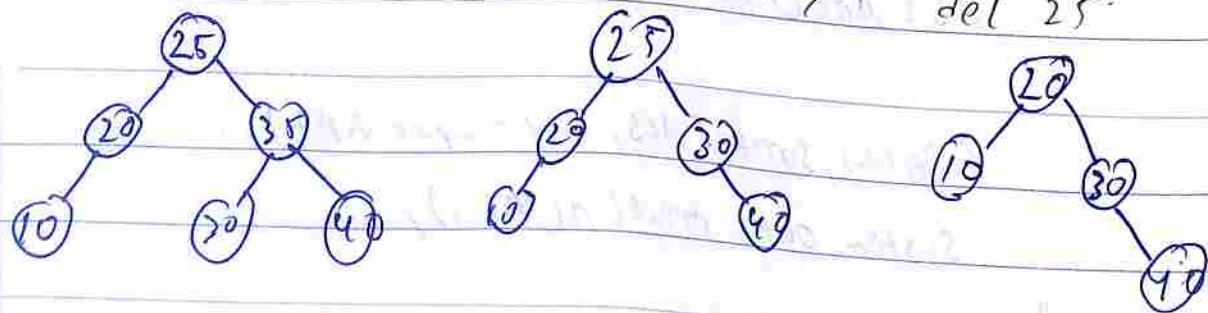
25, 15, 35, 10, 20, 30, 40.



del 15

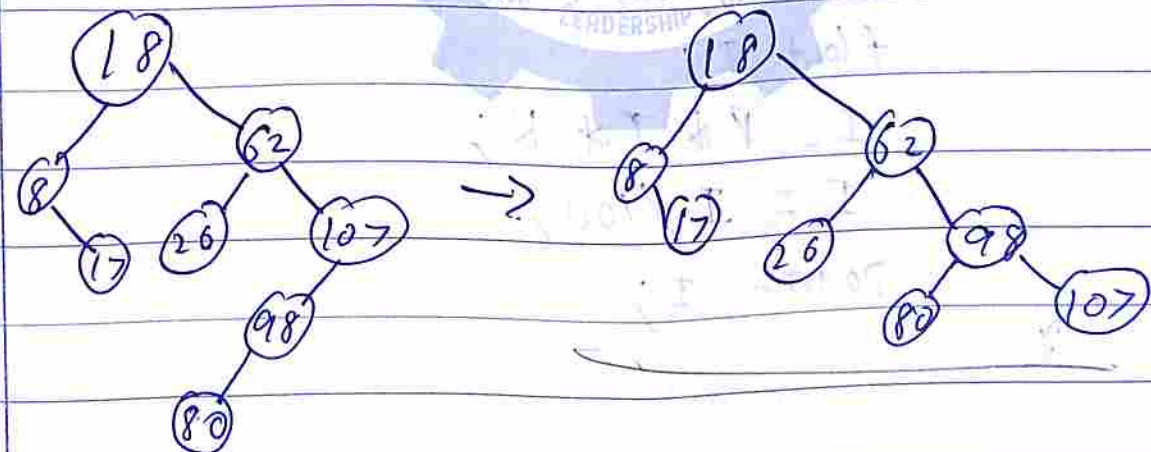
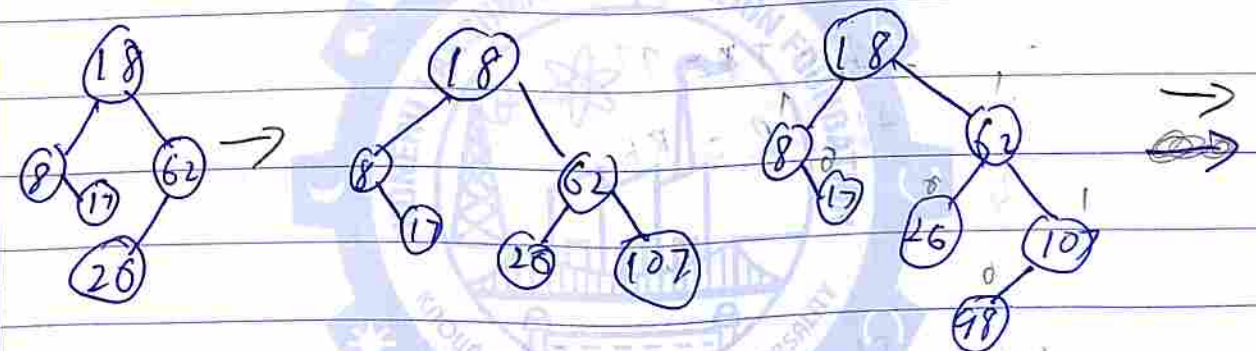
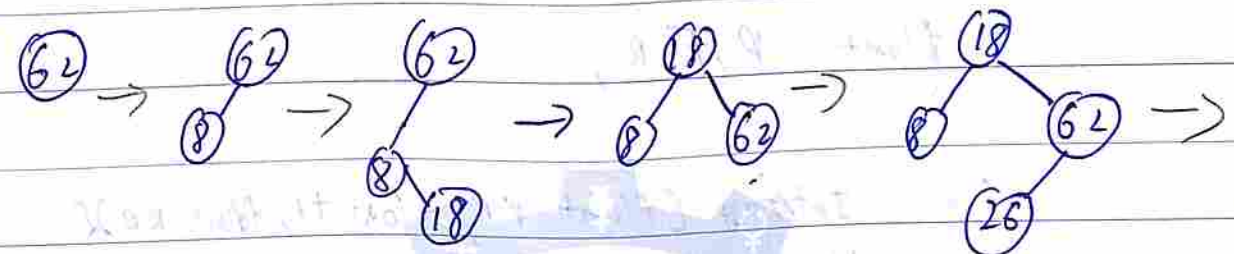
del 35

del 25



U.B AVL.

62, 8, 18, 26, 17, 107, 98, 80



}

3.A Compound Interest Calculator.

Public class Interest d.

float P, F, R;

class Interest (float PP, float TT, float RR) {

this.P = PP;

this.T = TT;

this.R = RR;

}

Public float calcInst() {

float I;

I = P * T * R;

I = I / 100;

return I;

}

}

3.B Chain of Responsibility

interface:

class Handler d.

public Successor Handle();

}

Class Concrete Handler1 implements Handler {

public Successor Handle Request() {

return 1;

}

}

Class Concrete Handler2 implements Handler {

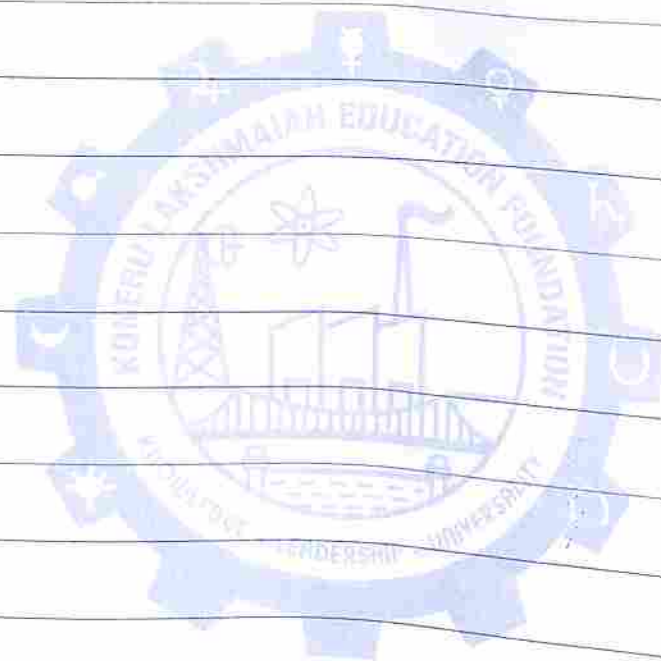
public Successor Handle Request() {

return 2;

}

}





$H_5 = \text{new } H_3 \angle \text{int } \text{cyl} \text{ , } f \text{ factor}$

